

Index Mechanisms

Inverted Index

How the Search Index Works: An Overview

Modern search engines do not scan documents one by one. Instead, they use an **Inverted Index**—a system that flips the relationship between files and text. Instead of asking "What words are in this document?", the index asks, "Which documents contain this word?"

To make searching massive datasets nearly instantaneous while keeping memory usage low, the index is split into three main components and uses an immutable segment architecture.

1. The Core Components

- **The FST (Term Index):** A highly compressed, tree-like structure that lives entirely in fast RAM. It stores the prefixes of words and acts as a map, telling the system exactly where to look on the hard drive.
- **The Term Dictionary:** A disk-based file containing the full list of sorted words. The FST points here. Once the exact word is found, the dictionary provides a byte-offset pointer to the final destination.
- **The Postings List:** A compressed, disk-based file containing the actual arrays of Document IDs (and exact word positions) where a specific term appears.

Building the Map (Sort-Based Inversion)

1. **Parsing:** As a document comes in, the engine assigns it a sequential integer (`DocID 1`). It tokenizes the text into words and spits out raw pairs: `(word, DocID)` .
2. **Spilling to Disk:** These unsorted pairs are continuously flushed to disk in chunks.
3. **The Great Sort:** Once all documents are parsed, the database engine runs an external merge-sort on the disk files, ordering everything alphabetically by the word.
4. **Inversion:** Because the pairs are now sorted, all the `(apple, 1)` , `(apple, 8)` , `(apple, 45)` pairs are physically adjacent. The system reads them sequentially and collapses them into the final inverted format: `apple -> [1, 8, 45]` .

2. The Operations Lifecycle

Inserts (The Write Path)

When a new document is added, the system does not rewrite the massive main database files—doing so would destroy performance.

- **In-Memory Buffer:** New documents are parsed and instantly indexed into a small, temporary inverted index that lives entirely in RAM.
- **Immutable Segments:** Once the RAM buffer is full, it is flushed to disk as a brand new, self-contained mini-index called a **Segment**.
- **Tiered Merging:** Segments are read-only (immutable). To prevent the system from drowning in thousands of tiny files, a background process constantly merges small segments together into larger ones.

Selects (The Read Path)

When a user types a query (e.g., "fast fox"), the engine executes a rapid, multi-stage lookup:

1. **Routing:** The search is broadcast to all existing segments (both the big historical ones and the tiny new ones) simultaneously.
2. **RAM to Disk:** For each segment, the query hits the in-memory FST, which instantly calculates the disk location.
3. **Fetching & Intersecting:** The engine jumps to the Term Dictionary, grabs the pointer, and reads the Postings List. For exact phrase matches, it runs a rapid CPU algorithm to ensure the numerical positions of "fast" and "fox" are right next to each other.
4. **Scoring:** The valid results from all segments are merged, scored for relevance, and returned to the user.

Deletes & Updates (The Tombstones)

Because segments are immutable, you cannot open a file to physically erase or modify a document.

- **Deletes:** If a document is deleted, the engine simply adds its ID to a separate file called a **Tombstone** (or Deletion Vector). During a search, if a document matches but is listed in the Tombstone, it is silently hidden from the results. The actual physical deletion happens later during the background merging process, where deleted documents are simply left out of the new, larger segment.
- **Updates:** In a search index, an "update" does not exist. Updating a document actually consists of two sequential operations: adding the old document ID to the Tombstone file (Delete), and inserting the modified document as a brand new entry (Insert).

You might also wonder, how the heck do we save every single word in all documents, isn't that too big? Well for that, you might want to have a look at Heaps' Law :)

B+ Trees

B+ Tree organizes data as a balanced tree where every node holds multiple keys. Internal nodes act purely as a routing map, they store keys only to direct searches left or right. All actual data lives exclusively in the leaf nodes (or the leafs point to the location on disk), which are linked together in a chain. When you search, you travel from root to leaf following

key comparisons. When you insert and a node gets too full, it splits into two and pushes a key up to the parent. This splitting can cascade upward, which is the only way the tree ever grows taller. Because all leaves are linked, range queries are a single traversal across the leaf chain without ever going back up the tree.

Extendible Hashing

Extendible Hashing uses a directory - an array of pointers - and a concept of bit depth to route keys to buckets. A global depth determines how many bits of each key's hash value you read to pick a directory slot. Each bucket has a local depth, which may be smaller than the global depth. When a bucket overflows, only that bucket splits: its local depth increases by one, a new bucket is created, its keys are redistributed, and the directory doubles only if the bucket's local depth now exceeds the global depth. This means growth is always incremental, you never rehash the entire table, only the one full bucket. If duplicates overwhelm the index that a page can't be split, we can use overflow buckets chained to the original bucket.

R-Trees

An R-Tree indexes geometric objects — points, rectangles, polygons — by wrapping groups of them in a Minimum Bounding Rectangle (MBR). The math that makes this possible is surprisingly simple: an MBR is just the tightest axis-aligned rectangle that contains a set of points or shapes. For a set of points, you compute it by taking the minimum and maximum value along each dimension independently — $MBR = [\min(x), \max(x)] \times [\min(y), \max(y)]$. For a group of existing MBRs, you apply the same idea one level up: the bounding rectangle of a set of rectangles is just the min of their left edges, the max of their right edges, the min of their bottom edges, and the max of their top edges. This composability is what makes the whole tree possible — MBRs of MBRs are still just MBRs.

Overlap and containment checks between two MBRs are equally simple. Two rectangles A and B overlap if and only if $A.\min X \leq B.\max X$ AND $A.\max X \geq B.\min X$ AND $A.\min Y \leq B.\max Y$ AND $A.\max Y \geq B.\min Y$ — six comparisons, no trigonometry, no floating point complexity. This is why MBRs are so practical: the spatial reasoning reduces to integer or float comparisons that are extremely cheap for a CPU.

To search, you start at the root and check which MBRs overlap your query region using that same formula, descending only into branches whose bounding box intersects. Because MBRs can overlap, you may need to explore multiple branches. Insertion places the object in whichever existing leaf causes the least MBR enlargement — computed as the difference in area before and after expanding the MBR to include the new object, where area is simply $(\max X - \min X) \times (\max Y - \min Y)$. When a node overflows, it splits and its parent MBR is recalculated bottom-up to reflect the new boundaries. The entire structure is held together by nothing more than min, max, and area — which is exactly why it scales to GPS systems, collision detection engines, and map databases without breaking a sweat.

Skip Lists

A Skip List is a linked list with multiple levels of express lanes stacked on top. The bottom level contains every element in sorted order. Each higher level contains a random subset of the level below, with each element being promoted to the next level by a coin flip. To search, you start at the top-left and move right as long as the next key is smaller than your target, then drop down a level when you overshoot. This combination of horizontal traversal and vertical dropping zeros in on the target in $O(\log n)$ time on average. Insertion finds the correct position via the same traversal, then randomly determines how many levels the new node gets promoted to.

LSM Trees

1. The Write (Memory & Durability)

When a new piece of data or an update arrives, two things happen simultaneously:

- **Write-Ahead Log (WAL):** The data is appended to a continuous file on disk to ensure it isn't lost if the power fails.
- **The MemTable:** The data is inserted into an in-memory data structure that keeps the keys sorted. While production systems often use Skip Lists for concurrent access, **this MemTable absolutely be implemented as a simple sorted in-memory array, an AVL tree, or a Red-Black tree**, depending on your specific performance needs and concurrency requirements.

2. The Flush (Moving to Disk)

Once that in-memory MemTable reaches a certain size threshold, it is frozen and temporarily becomes read-only, while a new MemTable is created to catch incoming writes. The frozen MemTable is then flushed sequentially to disk as an **SSTable (Sorted String Table)**. Because the data was already sorted in memory, this is a very fast, continuous disk write. The old WAL can now be safely deleted.

3. The Read Path

To find a record, the system searches in order of newest data to oldest:

- First, it checks the active MemTable.
- Next, it checks any frozen MemTables waiting to flush.
- Finally, it searches the SSTables on disk, starting with the most recently created ones. To avoid wasting time reading disk files that don't contain the target data, the system checks an in-memory **Bloom Filter** for each SSTable, which instantly confirms if the file is worth searching.

4. The Deletion (Tombstones)

Because SSTables are completely immutable (they cannot be edited once written), you cannot simply erase old data. Instead, deleting a record means writing a special marker called a **Tombstone** into the active MemTable. When the system executes a read and encounters this Tombstone, it immediately stops searching and returns a "Not Found" result, effectively masking any older versions of that data sitting deep on the disk.

5. The Maintenance (Compaction)

Over time, the disk fills up with hundreds of fragmented SSTables containing obsolete data and Tombstones. A background process called **Compaction** continuously gathers several older SSTables, merges them together, purges the dead Tombstoned records, and writes out a single, heavily optimized new SSTable. This keeps the total number of files low and ensures read speeds don't degrade over time.